Contents

Update an installed App to a newer version: `updateApp`

1. Problem
2. Use cases
3. Affected domains
4. Approach options
5. Recommended option
6. Flow and failure rules
7. Store and command shape
8. Test plan
9. Design review
10. Open questions

Update an installed App to a newer version: `updateApp`

Author: Steve · IDEA Lead Bot **Date:** 2026-10-28 **Status:** Draft (design review done: Console, Engine, App and Ops)

Covers agent-engine-dev#230 (approved to build by Koen, 2026-10-28) and the `upgradeApp` half of idea#76. Touches idea#74 (Upgrade Disk) and idea#77 (Engine self-upgrade). Code read at Engine main `2583608`.

1. Problem

An App instance is frozen at the version it was installed with. Once Kolibri 0.15.5 runs on a school's App Disk, there is no supported way to move that instance to a fixed or newer App version. The only options are:

- **Hand-edit** `/instances/<id>/compose.yaml` over SSH. This is what happened on the kit Engines on 2026-10-17 (the Kolibri Morango ID fix, app-kolibri#8), as a one-time exception.
- **Install a fresh instance** from the new version. This leaves the teacher's data in the old instance, and a duplicate name next to it (there's no remove command yet, idea#347).

Neither works in a school with no internet and no IT staff. Meanwhile the App Dev Bot ships App fixes every week (`app-update` issues, idea#88). Today they can't reach an installed school.

What the code already has: `upgradeApp` is an `OperationKind` with recovery `'fail' // not yet implemented` (`Operations.ts:38`). `isMajorUpgrade` / `extractMajorVersion` exist in `App.ts:47-65` and are unit-tested, but nothing calls them. The bundle (`apps/<appId>/`), the instance folder (`instances/<id>/` with `compose.yaml` , `.env` , `data/`) and the saved service images (`services/*.tar`) all live on the App Disk. `backupApp` / `restoreApp` (Borg) and `ResourceLock` already exist.

2. Use cases

1. **Field staff bring a newer App Disk.** A Kolibri 0.15.6 App Disk is docked next to the school's own disk, which runs Kolibri 0.15.5 with a year of learner progress. They want the school's instance on 0.15.6, keeping all its data. *(The core case; offline.)*
2. **A bug fix for an App that's already installed.** Same as 1, but the fix is a compose/env change (as app-kolibri#8). The source can be a docked disk, or GitHub when the Pi happens to be online (dev pool, Tailscale-reachable kits).

3. **The teacher sees "Update available"** in the Console after a newer disk was docked, and taps one button. No terminal, no choices beyond "Update" or "Not now". *(This is #76's dock-time part, built on the same command.)*
4. **The update goes wrong.** The new version won't start, or starts but serves errors. The school must end up on the old version with its data unchanged, without anyone touching it.
5. **A major version jump** (Nextcloud 27 to 29, Kolibri 0.x to 1.x) needs the App's own data migration, which may be one-way.

Constraints from these: offline sources first; one tap; never lose data; failure is undone automatically; the Pi is slow and the disk can be pulled at any moment.

3. Affected domains

Domain	Change
Engine (Axle)	`updateApp` command and `upgradeApp` operation; pre-update snapshot; health gate; rollback; dock-time "update available" detection (#76).
Console (Pixel)	"Update available" badge from `updateAvailable`, shown only when the Engine has the `updateApp` capability; confirm, progress and result dialogs (§7.3).
App Disks / Kid	`x-app` fields (`healthCheck`, `healthTimeout`, `updatesFrom`, `migrationNote`, `snapshotExclude`) in app-kolibri, app-nextcloud, app-kiwix and app-milkwise; a harness test "update from previous release with real data"; measured first-start times after migrations on a pool Pi to set the defaults.
Ops (Atlas)	Reviewed: disk space, backups, reboot safety, fleet rollout (§9).

4. Approach options

4.1 Where the new version runs

A. In-place swap. Stop the instance, keep `data/` and `.env`, replace the bundle-derived files (`compose.yaml` and the other bundle files, `services/*.tar`), re-apply the Engine's compose adjustments (`x-app.version`, `instanceName`, port, files/usb mounts), start. - Pros: same instance ID, name, port, URL, backups and peer references; no extra disk space for data; matches #230's proposal. - Cons: the old version must be kept somewhere for rollback; an App that migrates `data/` on first start changes it in place.

B. Side-by-side new instance. Install the new version as a second instance, copy `data/` into it, start it, then retire the old one. - Pros: the old instance stays untouched until the switch, so rollback is trivial. - Cons: doubles the data on disk (Kolibri content can be tens of GB on a Pi SSD); a new instance ID breaks backup history, bookmarks and the Console's identity; two instances with the same port/hostname collide (the issue behind #230); needs remove (idea#347) to finish.

4.2 Protecting the data

- **None.** Unacceptable for case 4 and 5.
- **Full Borg backup** before update (`backupApp`). Strong, but needs a docked Backup Disk and is slow on a Pi.
- **Local snapshot**: before the swap, move the old bundle files to `instances/<id>/.pre-update/<oldAppId>/` and copy the instance's data there: `data/` **and its Docker named volumes** (for example Nextcloud's database volume), minus anything the App lists in `x-app.snapshotExclude` (for example Kolibri's `content/`, which is re-importable and large). Fast to restore, but costs disk space. The free-space check measures exactly this set plus a margin; without room the update is refused (`NO_SPACE`).

4.3 Deciding the update worked

The Engine already sees "containers running". That isn't enough: Kolibri can run and still return 500 after a failed migration. The new version passes only if, within `x-app.healthTimeout` (default 10 min on a Pi): 1. the primary service reports **healthy** in Docker; then 2. the `x-app.healthCheck` URL (else the instance's main URL) returns 2xx/3xx and, if given, contains the expected body string (Nextcloud `status.php`: `"maintenance":false` and `"needsDbUpgrade":false`).

A climbing container restart count (crash loop) fails the gate early instead of waiting out the timeout. Any failure: automatic rollback.

4.4 Minor vs major

- **Minor** (same major, `isMajorUpgrade` false): data format is compatible or the App migrates forward safely. Allowed from the Console and at dock time.
- **Major**: migrations may be one-way and slow. Allowed only if the new App declares the old major in `x-app.updatesFrom` (Nextcloud lists only its previous major, because Nextcloud can't skip majors), and only with a Borg backup made in the last hour, because rollback then means restoring the snapshot, not just the old images.

4.5 Relation to dock-time upgrade (#76), #74 and #77

76 has two halves: *detect* a newer version when a disk is docked, and *perform* the upgrade. The second is exactly `updateApp`. Build the command first; then the dock path only writes an "update available" hint, and only when the newer bundle is on a disk docked on the same Engine as the instance. The Console offers the command. The Engine never updates on its own at dock time, because a teacher may dock a disk only to copy something.

The Upgrade Disk (#74) and Engine self-upgrade (#77) are about the Engine, not Apps. They can reuse the same pattern (snapshot, health gate, rollback, `Operation` record), but they're not part of this change.

5. Recommended option

In-place swap (A), with a local snapshot, a health gate and automatic rollback; minor versions from the Console and dock-time hints; major versions only when the App opts in and a fresh Borg backup exists.

Why: the instance identity (ID, name, port, backups) is what the school and the other Pis know, and A keeps it. B's double data and duplicate-name problems are exactly what #230 is trying to remove. The local snapshot gives B's "old one is untouched" safety at the cost of one data copy that's deleted after success. The health gate turns "it started" into "it works", which is what makes automatic rollback safe without IT staff.

6. Flow and failure rules

1. **Check** (no changes). Any failure ends with a refusal code and nothing changed: `NOT_DOCKED` (instance's disk not docked here), `BUSY` (operation or `ResourceLock` on instance or disk), `SOURCE_MISSING` (no readable `apps/<newAppId>/` with its images), `DIFFERENT_APP` (other App name), `NOT_NEWER`, `MAJOR_NOT_ALLOWED` (not in `updatesFrom`, or no `--allow-major`), `NEEDS_BACKUP` (major without a Borg backup under 1 h old), `NO_SPACE` (§4.2).
2. **Journal and snapshot**: write the journal to `.pre-update/` on the disk; stop the instance and wait until its containers are really gone (fixes the stop/start race in #230); move the old bundle files there;

copy `data/` and the named volumes (minus `snapshotExclude`).

3. **Swap**: copy the new bundle into the instance and re-apply the Engine's compose adjustments with the **same code `installApp` uses** (`.env` kept: port, pass, hostname, ip; `x-app.version`, `instanceName`, files/USB mounts); load the new image tars; set `instanceOf` to the new App ID. Old images stay in Docker.
4. **Start and health gate** (§4.3).
5. **Success**: operation Completed; trace reports old → new. 24 h after the gate passed, the snapshot moves to quarantine (never deleted by the Engine). The old bundle `apps/<oldAppId>/` stays on the disk.
6. **Failure in 3 or 4**: stop; restore the old bundle **and the data copy** (`data/` and volumes) from the snapshot; restore `instanceOf`; start the old version (its images are still in Docker); operation Failed with `ROLLED BACK: <why>`. If rollback itself fails, leave the instance Stopped with a `statusCondition` explaining it and the snapshot intact.
7. **Disk pulled or Engine restarted mid-update**: the journal on the disk is the source of truth; the store operation is only the visible record. Whichever Engine docks that disk next sees the unfinished journal and rolls back (step 6) **before anything on the disk starts**. Recovery strategy for `upgradeApp` stays `'fail'`.

7. Store and command shape

7.1 Command

```
updateApp <instanceIdOrName> <newAppId> [--source <diskId>] [--allow-major]
```

- Scope `engine`; resolved with `resolveInstanceArg` / `resolveDiskArg` (IDs, never disk names). Without `--source`, it uses a disk docked on this Engine that has `apps/<newAppId>/`.
- GitHub as a source (when online) is a dev/ops command option only, for tagged releases; the Console never offers it.
- New Engine capability `updateApp`, so the Console knows the command exists.

7.2 Store

- Operation: `kind: 'upgradeApp'`, subject = instance, args `{fromVersion, toVersion, major, estimatedMinutes, healthTimeoutMs, sourceDiskId}`, steps Check / Snapshot / Swap / Start / Health, with the existing `currentStep` / `stepLabel` on the instance.
- Instance: `instanceOf` changes only on success; new optional `previousAppId?: AppID | null` ("Updated from 0.15.5").

- `instance.updateAvailable?: { appId, diskId, fromVersion, toVersion, major, estimatedMinutes } | null`. Set only when the newer bundle is on a disk docked on the same Engine; cleared on that disk's undock, on Engine boot, and when the update finishes. `estimatedMinutes` = snapshot copy estimate + health timeout.
- App bundle (`x-app`): optional `healthCheck: {path, expect?}`, `healthTimeout`, `updatesFrom: [<major>…]`, `migrationNote`, `snapshotExclude: [<path>…]`.

7.3 Console (Pixel)

- Badge **Update available** from `updateAvailable`, only if the Engine has the `updateApp` capability.
- **Not now** hides the badge on that browser only; nothing is written to the store.
- Confirm: "Update Kolibri from 0.15.5 to 0.15.6? Your data is kept. A copy is made first. This takes about N minutes, and Kolibri is unavailable meanwhile." For a major, the App's `migrationNote` is shown, and **Update** is disabled with "Make a backup first" when there is no Borg backup under 1 h old.
- Progress: the step label (Checking, Making a copy, Updating, Starting, Checking it works).
- Result: "Kolibri is updated to 0.15.6", or "The update didn't work, so Kolibri is back on 0.15.5 with all its data. Reason: …". Refusal codes map to plain sentences.

8. Test plan

- **Unit**: argument parsing; every refusal code; minor/major rule; free-space measurement incl. named volumes and `snapshotExclude`; health gate (Docker healthy, body string, crash-loop early fail); `.env` and Engine-adjustment merge; rollback on every failing step (mocked docker).
- **Engine integration (box, test mode)**: sample App 1.0 → 1.1 with data kept; 1.1 with a broken health URL rolls back to 1.0 with the same data; refusal when busy, when the disk is missing, for a major version without opt-in.
- **Pi (pool Pi, claimed)**: real Kolibri and Nextcloud (DB volume) updates with real data; pull the disk during Swap and re-dock on **another** Engine, check it rolls back before starting; reboot during Start, check rollback on boot; Engine restarted during the health gate; `updateAvailable` cleared on undock and boot.
- **Console**: "Update available" badge, confirm, progress and the rolled-back message (mock store + Playwright).
- **App (Kid)**: harness test "update from previous release with real data" for app-kolibri, app-nextcloud, app-kiwix, app-milkwise; measured first-start times after migration set each App's `healthTimeout`.

9. Design review

- **Console (Pixel):** badge gated on the capability, local-only "Not now", confirm/progress/result wording, migration note and the backup gate for majors (§7.3).
- **Engine (Axle):** named volumes in the snapshot and the space check, the on-disk journal with rollback by whichever Engine docks next, `updateAvailable` scoping and fields, Docker-healthy plus body-string health gate with crash-loop early fail, rollback of data (not just images), reuse of the install compose code, refusal codes and the `updateApp` capability.
- **App Disks (Kid):** Nextcloud `updatesFrom` limited to the previous major, `snapshotExclude` for Kolibri `content/`, the `x-app` fields in four App repos, the harness update test and measured timings.
- **Ops (Atlas):**
- *Space:* the free-space check covers both disks: the App Disk (copy of `data/` and the named volumes minus `x-app.snapshotExclude`, plus the new image tars, plus about 10% margin) and the system disk's `/var/lib/docker`, because old and new images sit side by side. No hardlink (`cp -al`) snapshots, since SQLite and MariaDB change files in place. Old images are cleaned up only when the snapshot is deleted (moved to quarantine), not before.
- *Backups:* the `NEEDS_BACKUP` refusal text says "Dock a Backup Disk and make a backup first". In the field this effectively means no major updates without a Backup Disk, which is accepted.
- *Reboot safety:* add an integration test "Engine restarted during the health gate" next to the disk-pull test (pool Pis and the `idea` Pi reboot daily at 05:00). The Engine logs the CPU temperature during the swap. Fleet runs keep the 75 °C pause rule.
- *Rollout:* no new sudoers or root helpers (`docker load` and compose already run as pi). Order: Engine PR tested on claimed pool Pi idea04, then merge, then deploy to idea01, then golden. Kits only in an overnight window Koen opens.
- *Open questions:* Atlas's answers agree with the decisions below (per-App `healthTimeout` default 10 min; Borg backup required for majors; refuse without a copy; GitHub update is ops/dev-only; one instance at a time, later Pi by Pi stopping at the first rollback; keep the old bundle until the snapshot is deleted).

Decided in review (former open questions): health timeout per App with a 10 min default; a major update needs a Borg backup under 1 h old; no space means refuse (`NO_SPACE`); GitHub updates only as a dev/ops command for tagged releases; one instance at a time; the old bundle stays on the disk.

10. Open questions

All four reviews are done. Only one item remains:

1. **Measured timings (Kid):** Kolibri 0.15.5 to main with real data and Nextcloud one major step, measured on idea04: first-start times after migration, to set each App's `healthTimeout` and the `estimatedMinutes` copy rate.